

SQL

1. History

2. The Workbench

3. SQL Basics

4. SQL Statements

5. SHOW

6. SELECT

1. DISTINCT

2. WHERE

3. AND / OR / NOT

4. ORDER BY

5. NULL

6. LIKE

7. IN

8. BETWEEN

9. LIMIT

7. Aggregate Function

1. MIN / MAX

2. COUNT / AVG / SUM

3. GROUP BY

4. HAVING

8. Combining Tables

8. JOIN

9. UNION

9. Views

10. Stored Procedures

11. Multiple databases

12. Creating Databases

1. Normal Approach

2. Create Database

3. Creating Tables

4. Removing Tables / Data

5. Changing Tables

6. if exists

13. Constraints

14. Filling Tables

15. UPDATE

16. DELETE

17. Definitions

18. Literature

History

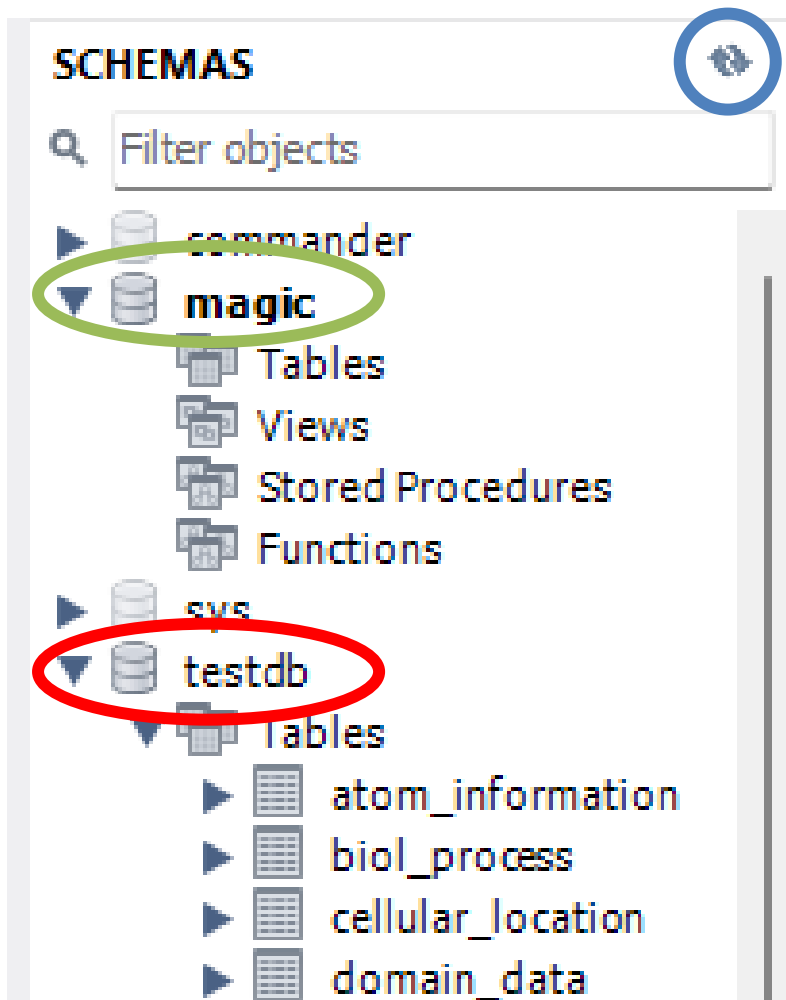
- Was initially developed at IBM by Donald D. Chamberlin and Raymond F. Boyce after learning about the relational model from Edgar F. Codd
- Initially called SEQUEL (Structured English Query Language), was designed to manipulate and retrieve data stored in IBM's original quasi-relational database management system, System R
- The original name SEQUEL, which is widely regarded as a pun on QUEL, the query language of Ingres was later changed to SQL because "SEQUEL" was a trademark of the UK-based Hawker Siddeley Dynamics Engineering Limited company

- Commercial products based on their System R prototype, including System/38, SQL/DS and IBM Db2, were made commercially available in 1979, 1981, and 1983, respectively
- In June 1979, Relational Software introduced one of the first commercially available implementations of SQL, Oracle V2 for VAX computers
- By 1986, ANSI and ISO standard groups officially adopted the standard "Database Language SQL" language definition
- New versions of the standard were published in 1989, 1992, 1996, 1999, 2003, 2006, 2008, 2011 and most recently 2016

The Workbench

- During this course we will work with the freely available MySQL workbench and a locally installed MySQL server
- You will use a database I prepared in form of a SQL script, so that everyone of you has the same tables and data available
- This workbench is only one of many different RBDMS's which are available, so focus more on the SQL syntax when on the functionality of this program
- During this course you can write your SQL statements in a query editor and save them in form of SQL scripts, so that you have them available all the time

- While we can do nearly everything with the help of SQL statements we can also make use of the GUI of our workbench
- That means I will show you how to maneuver through the workbench with the help of mouse and keyboard
- Especially when creating a database including the corresponding tables, it's much easier to do so with the help of mouse and keyboard



- The workbench contains a sidebar on the left, where we can see all databases we have available, for example a database testdb (red circle)
- Before we can work on a database we need to activate it by double clicking on its name, and afterwards the name appears in bold (green circle)
- Whenever you create something, you need to refresh the sidebar with the small blue button that is composed out of two arrows (blue circle)

SQL Basics

- A standard language for storing, manipulating and retrieving data in databases
- Stands for Structured Query Language
- Lets you access and manipulate databases
- Although SQL is an ANSI/ISO standard, there are different versions of the SQL language
- However, to be compliant with the ANSI standard, they all support at least the major commands (such as SELECT, UPDATE, DELETE, INSERT, WHERE) in a similar manner

- SQL can execute queries against a database
- SQL can retrieve data from a database
- SQL can insert records in a database
- SQL can update records in a database
- SQL can delete records from a database
- SQL can create new databases and delete databases
- SQL can create and delete tables in a database
- SQL can create stored procedures in a database
- SQL can create views in a database
- SQL can set permissions on tables, procedures, and views

- Every table is broken up into smaller entities called fields
 - A field is a column definition in a table that is designed to maintain specific information about every record in the table
 - A field therefore is the column header or column definition
- A record, also called a row, is each individual entry or dataset that exists in a table
 - A record is a horizontal entity in a table
- A column is a vertical entity in a table that contains all information associated with a specific field in a table

- To avoid any confusion when talking about fields we will stick to the following annotation:
 - ▶ The definition of a column will be called Header by me
 - ▶ A Cell on the other hand refers to the smallest unit of a table and contains the value associated to one row and one column at the same time

The diagram illustrates the components of a database table. A table with 6 columns and 3 data rows is shown. The first column, 'Identifier (string, PK, FK)', is circled in red and labeled 'Header' with a downward arrow. The second column, 'Resolution (float)', is labeled 'Column' with a downward arrow. The first row, containing '5SY1', is labeled 'Row' with a rightward arrow. The cell containing '2014-12-03' is circled in red and labeled 'Cell' with an upward arrow. A bracket on the right side of the table is labeled 'Table'.

Identifier (string, PK, FK)	Resolution (float)	R_Free (float)	Released (Date)	Clashscore (int)	Ram_outl (float)
5SY1	3.9		2016-08-24	6	0.10
5JMB	2.05	0.231	2016-06-29	2	0.00
3WZS	1.7	0.247	2014-12-03	0	0.00

SQL Statements

- Most of the actions you need to perform on a database are done with SQL statements which is the SQL term for commands / functions
- The following SQL statement for example shows all the rows in the “proteins” table:
 - ▶ `SELECT * FROM proteins;`
- SQL keywords are NOT case sensitive: select is the same as SELECT
 - ▶ But you should write SQL statements with all capital letters since this is the international convention
- Table & column names as well as the data in the tables are also case in-sensitive
- This case insensitivity is a property of MySQL

- Some database systems require a semicolon at the end of each SQL statement
- Semicolon is the standard way to separate each SQL statement in database systems that allow more than one SQL statement to be executed in the same call to the server
- In our workbench we always need a semicolon “;” at the end of our statements to signal SQL the end of them
- Since SQL recognizes everything (except comments) as part of one SQL statement as long as it doesn’t encounter a semicolon, we can write our statements over multiple lines to increase the readability of them

- Comments are used to explain sections of SQL statements, or to prevent execution of SQL statements
- Single Line Comments:
 - ▶ Single line comments start with `--` and are followed by a space
 - ▶ Any text between `--` and the end of the line will be ignored (will not be executed)
 - ▶ The following example uses a single-line comment as an explanation:

```
-- Select all:
```

```
SELECT * FROM proteins;
```

- Multi-line Comments:

- ▶ Multi-line comments start with `/*` and end with `*/`
- ▶ Any text between `/*` and `*/` will be ignored
- ▶ The following example uses a multi-line comment as an explanation:

```
/*Select all the columns  
of all the records  
in the proteins table:*/  
SELECT * FROM proteins;
```

SHOW

- The statement SHOW enables you to display different things of your database / server
- We can use it to display all available databases (including some hidden ones) on our server by using it as follows:

```
SHOW databases;
```

- We can use it to display all available tables in the current active database by using it as follows:

```
SHOW tables;
```

- Last but not least we can use it to show an overview of all column definitions of a specific table in our database by using it as follows:

```
SHOW columns FROM table_name;
```

- So to show the column definitions of the table proteins I can use:

```
SHOW columns FROM proteins;
```


The SELECT statement

- The SELECT statement is used to filter data from a database
- The data returned is stored in a temporary result table, also called the result-set
- When working with SELECT we have to specify the table we want to get data from as well as the columns we want to be part of our result table
- That means all SELECT statement could be read as give me the columns ... from the table ...
- SELECT works on rows, that means we go through the table we want to work on and pick all rows where the data in them fulfills certain criteria

- The SELECT statement is the most basic one and its functionality can be expanded by adding other statements to it
- You can also combine multiple SELECT statements in form so called sub queries to make use of relations between tables

- General Syntax:

```
SELECT column1, column2, ...
```

```
FROM table_name;
```

- The following SQL statement shows the columns “Protein_Name” and “Protein_Length” columns from the “proteins” table in the result table:

```
SELECT Protein_Name, Protein_Length FROM proteins;
```

- If we want to look at all columns of the table proteins we can use the wildcard * to replace the column names in the SELECT statements:

```
SELECT * FROM proteins;
```

- Before we start with our first exercise, some general information about them
 - ▶ You can finish all of them with the help of the statements you got to know until that point
 - ▶ The tables you need to work on are in most cases written in red, e.g. **organisms**
- All exercises have an optional part, you can tackle after you finished the main part
 - ▶ These optional parts can be more complex and are not as descriptive as the main part

- When working with SELECT statements we have to stick to a certain Order
- While we can leave out many of the different statements, when we use them we have to place them at the correct spot:
 - ▶ SELECT
 - ▶ FROM (JOIN ... ON)
 - ▶ WHERE
 - ▶ GROUP BY
 - ▶ HAVING
 - ▶ ORDER BY
 - ▶ LIMIT

- Write SELECT statements that fulfill the following criteria:
 - ▶ The Name and taxonomic name of all **organisms**
 - ▶ All information about **kingdoms**
 - ▶ The resolution, the R-free value and the Clashscore for **structural_data**
 - ▶ All information about **secondary_structure**
- Optional:
 - ▶ Charge, Mass and simplified formula for small molecules
 - ▶ Weight, Melting Point and IUPAC-Name for non-canonical amino acids

- The SELECT DISTINCT statement is used to return only different values from a specific column
- Inside a table, a column often contains duplicate values, especially when working with foreign keys
- Sometimes you only want to list the different (distinct) values and in these cases we can use the statement DISTINCT to do so
- Syntax:

```
SELECT DISTINCT column_name
```

```
FROM table_name;
```


- Furthermore you can use SELECT DISTINCT with more than one column to get only the unique combinations of the values in these columns
- Syntax:

```
SELECT DISTINCT column1,column2,...
```

```
FROM table_name;
```

- The following SQL statement selects only the DISTINCT values from the "Organism_ID" column in the "proteins" table:

```
SELECT DISTINCT Organism_ID FROM proteins;
```

- For getting back all unique combinations of the values from columns Source_ID and Method_ID in the table structure we could use the following statement:

```
SELECT DISTINCT Source_ID, Method_ID FROM structures;
```

- Write SELECT statements that fulfill the following criteria:
 - ▶ For which Kingdom_IDs do you have **organisms** in the database?
 - ▶ For which Structure_IDs do you have data in **secondary_protein**?
 - ▶ For which Method_IDs do you have **structures** in your database?
 - ▶ For which Protein_IDs do you have data in **modifications_proteins**?
- Optional:
 - ▶ For which proteins can you find structures in the database?
 - ▶ For which hetero atoms do you have IUPAC-Names in the database?
 - ▶ What are the different maximal repeats you can find for domains in the database?

- The WHERE clause is used to filter records / rows and thus helps us extract only those records / rows that fulfill a specified condition
- We can use many different clauses and combine them with the help of logical operators
- When using WHERE we check the condition for each row of our table and only if the data in a row matches the condition it becomes a part of our result table
- Syntax:

SELECT column_name(s)

FROM table_name

WHERE condition;

Operator	Description
=	Equal
>	Greater than
<	Less than
>=	Greater than or equal
<=	Less than or equal
<>	Not equal. Note: In some versions of SQL this operator may be written as !=

- When creating the conditions to work with WHERE we have different operations available to us
- The most basic ones are shown in the table above and we will talk about some more complicated ones, which work with additional statements later on

- All operations you saw on the previous slide work with numerical as well as string values
- If you use operators like $>$, $<$, $>=$, $<=$ on string values they're sorted in alphabetical order, with "a" being the "lowest" value and "z" being the "highest" value
- That means in a column with values "Helix", "Turn" and "Strand" and you want to search for values greater than "Helix" you would get back "Strand" and "Turn"

- When using conditions in our SELECT statements we always use the general syntax for them
- This syntax is as follows:
 - column_name operator value
- The column_name needs to be a column of the table our SELECT statement works on
- The operator can be things like = or > as well as statements like IN or BETWEEN
- The value represents the value we compare the data in the column with

- The following SQL statement selects all the rows with the name “Helix”, in the “secondary_structure” table:

```
SELECT * FROM secondary_structure  
WHERE Structure_Name='Helix';
```

- MySQL requires single quotes around text values (most database systems will also allow double quotes)
- When selecting according to strings we check for an absolute match which is NOT case sensitive
- That means in the example below we only check whether or not the value in our row consists only of the letters h, e, l, i, x in this order

- However, numeric fields should not be enclosed in quotes:

```
SELECT * FROM proteins  
WHERE Organism_ID=1;
```

- Write SELECT statements that fulfill the following criteria:
 - ▶ All **proteins** with more than 1000 amino acids
 - ▶ All **structures** with a Source_ID of 1
 - ▶ All **structural_data** with a Resolution smaller than 2.0
 - ▶ All **organisms** that have a Kingdom_ID of 1
 - ▶ All **proteins** with a mass smaller than 25000
 - ▶ All **proteins** named 'Cytochrome c oxidase subunit 1'
- Optional:
 - ▶ Atoms with a positive charge
 - ▶ Atoms with a mass between 50 and 150 (you need a logical operator here, sorry)

- The WHERE clause can be combined with AND, OR, and NOT operators
- The AND / OR operators are used to filter records based on more than one condition:
 - ▶ The AND operator displays a record if all the conditions separated by AND are TRUE
 - ▶ The OR operator displays a record if any of the conditions separated by OR is TRUE
 - ▶ The NOT operator displays a record if the condition(s) is NOT TRUE

- Syntax

```
SELECT column_name(s)
```

```
FROM table_name
```

```
WHERE condition1 AND condition2 AND condition3 ...;
```

- The following SQL statement selects all rows from “proteins” where the value of Annotation is 1 and the Organism_ID is also 1:

```
SELECT * FROM proteins
```

```
WHERE Annotation=1 AND Organism_ID=1;
```

- Syntax:

```
SELECT column_name(s)
```

```
FROM table_name
```

```
WHERE condition1 OR condition2 OR condition3 ...;
```

- The following SQL statement selects all rows from “proteins” where the value of Annotation is 1 or the Organism_ID is 1:

```
SELECT * FROM proteins
```

```
WHERE Annotation=1 OR Organism_ID=1;
```

- Syntax:

```
SELECT column_name(s)
```

```
FROM table_name
```

```
WHERE NOT condition;
```

- The following SQL statement selects all fields from “proteins” where the value of Annotation is NOT 1:

```
SELECT * FROM proteins
```

```
WHERE NOT Annotation=1;
```

- When working with NOT keep in mind that only the condition directly after the statement NOT is negated
- That means the statement below will select all rows which have the value 1 in the Organism_ID column and not the value 1 in the Annotation column:

```
SELECT * FROM proteins
```

```
WHERE NOT Annotation=1 AND Organism_ID=1;
```

- If you want to negate both statement you would need:

```
SELECT * FROM proteins
```

```
WHERE NOT Annotation=1 AND NOT Organism_ID=1;
```

- By adding brackets to your statement you can adjust the behavior of NOT to negate combinations of conditions:

```
SELECT * FROM proteins
```

```
WHERE NOT (Annotation=1 AND Organism_ID=1) ;
```

- Of course you could use OR inside the brackets as well, depending on the logic you're searching for:

```
SELECT * FROM proteins
```

```
WHERE NOT (Annotation=1 OR Organism_ID=1) ;
```


- Write SELECT statements that fulfill the following criteria:
 - ▶ All **proteins** with more than 1000 amino acids and a Mass greater than 100000
 - ▶ All **structural_data** with a Resolution less than 2.0 or an R-free value smaller than 0.25
 - ▶ All **proteins** where the Organism_ID is not 4
 - ▶ All **organisms** with a Kingdom_ID of 1 or 2
 - ▶ All **proteins** with an Organism_ID of 3 or 28
- Optional:
 - ▶ Atoms with a positive charge, a mass higher than 100 and a CHEBI-ID over 20000
 - ▶ Structures generated with the Methods 1, 2 or 3
 - ▶ Modifications with a mass over 125, more than 4 Hydrogenbond-donors and -acceptors

- The ORDER BY statement is used to sort the result-set in ascending or descending order according to the values in a certain column
- The ORDER BY keyword sorts the records in ascending order by default
- To sort the records in descending order you need to add the DESC statement after the column name
- The column you want to use as basis for the sorting needs to be part of either the table your working on or the result table you will create with your SELECT statement

- Syntax

```
SELECT column_name(s)
```

```
FROM table_name
```

```
ORDER BY column (DESC);
```

- ORDER BY can also be used with multiple columns and in these cases the column you specified first is used as main sorting column and only if you have duplicate values in this column the second column is used and so on:

```
SELECT column_name(s)
```

```
FROM table_name
```

```
ORDER BY column1 (DESC), column2 (DESC), ...;
```

ID	Length	Mass
1	150	15623
2	250	27563
3	150	16753
4	250	25632
5	350	37466
6	300	33564

- Imagine the table at the top on the left side with this small datasets
- Now we can create a SELECT statement with the help of ORDER BY

```
SELECT * FROM Extable ORDER BY Length, Mass
```

ID	Length	Mass
1	150	15623
3	150	16753
4	250	25632
2	250	27563
6	300	33564
5	350	37466

- As you can see in the result table at the bottom on the left side the data is ordered first by Length and than by Mass

- The following SQL statement selects all rows from the “protein” table, sorted by the “Protein_Name” column:

```
SELECT * FROM proteins  
  
ORDER BY Protein_Name;
```

- Rows from the “protein” table, sorted by the DESCENDING “Protein_Name” column :

```
SELECT * FROM proteins  
  
ORDER BY Protein_Name DESC;
```

- The following SQL statement selects all rows from the “proteins” table, sorted ascending by the “Organism_ID” and descending by the “Protein_Name” column:

```
SELECT * FROM proteins
```

```
ORDER BY Organism_ID, Protein_Name DESC;
```

- We can use ORDER BY in combination with WHERE clauses as well:

```
SELECT * FROM proteins WHERE Organism_ID=1
```

```
ORDER BY Protein_Name DESC;
```

- Write SELECT statements that fulfill the following criteria:
 - ▶ All **proteins** ordered by length
 - ▶ All **proteins** ordered by Mass in desc. order
 - ▶ All **proteins** with Organism_ID 4 ordered by Annotation
 - ▶ All **structures** with Source_ID 2 ordered by their Identifier
 - ▶ All **organisms** ordered by taxonomic name in desc. order
 - ▶ All **structural_data** ordered by asc. Ramachandran outliers and desc. sidechain outliers
- Optional:
 - ▶ Non-canonic amino acids sorted by hydrogen bond donors and acceptors
 - ▶ All domains with at least two repeats sorted by their minimum length

- A cell with a NULL value is a cell with no value
- If a field in a table is optional, it is possible to insert a new record or update a record without adding a value to this field
- Then, the cell will be saved with a NULL value
- It is not possible to test for NULL values with comparison operators, such as =, <, or <>
- We will have to use the IS NULL and IS NOT NULL operators instead

- IS NULL Syntax:

```
SELECT column_name(s)
```

```
FROM table_name
```

```
WHERE column_name IS NULL;
```

- IS NOT NULL Syntax:

```
SELECT column_name(s)
```

```
FROM table_name
```

```
WHERE column_name IS NOT NULL;
```

```
SELECT Short, Gene_Ontology, Uniprot_ID  
FROM cellular_location  
WHERE Uniprot_ID IS NULL;
```

```
SELECT Short, Gene_Ontology, Uniprot_ID  
FROM cellular_location  
WHERE Uniprot_ID IS NOT NULL;
```

- Write SELECT statements that fulfill the following criteria:
 - ▶ All **structural_data** without a R-free value
 - ▶ All **structural_data** with a Resolution
 - ▶ All **domain_data** that don't have a maximum repeat
 - ▶ All **cellular_locations** that have a description
 - ▶ All **modification_data** that don't have an EC-Number
- Optional:
 - ▶ hetero atoms with a SMILES formula and a positive charge
 - ▶ modifications without a melting point and at least 3 hydrogen bond donors sorted by name
 - ▶ structures with R-free value and Resolution sorted by release date

- The LIKE operator is used in a WHERE clause to search for a specified pattern in a text column
- There are two wildcards often used in conjunction with the LIKE operator:
 - ▶ The percent sign (%) represents zero, one, or multiple characters
 - ▶ The underscore sign (_) represents one, single character
- Syntax:

```
SELECT column_name(s)
```

```
FROM table_name
```

```
WHERE column LIKE pattern;
```

```
SELECT * FROM proteins WHERE Protein_Name LIKE 'A%';
```

➔ Protein_Name starts with A

```
SELECT * FROM proteins WHERE Protein_Name LIKE 'a%';
```

➔ Protein_Name starts with a ➔ Patterns are case insensitive

```
SELECT * FROM proteins
```

```
WHERE Protein_Name LIKE '%A' OR Protein_Name LIKE '%s';
```

➔ Protein_Name ends with A or ends with s

```
SELECT * FROM proteins
```

```
WHERE Protein_Name LIKE '% RNA %';
```

➔ Protein_Name contains the word RNA with spaces before and after it

```
WHERE Protein_Name LIKE '%RNA%';
```

➔ Protein_Name contains the word RNA without spaces before and after it

```
SELECT * FROM proteins WHERE Protein_Name LIKE '_i%';
```

➔ Protein_Name has an i at the second position

```
SELECT * FROM proteins WHERE Protein_Name LIKE '____o%';
```

➔ Protein_Name has an o at the fourth position

- Write SELECT statements that fulfill the following criteria:
 - ▶ All **proteins** whose name starts with an “S”
 - ▶ All **organisms** whose taxonomic names contain “ano”
 - ▶ All **structures** with Identifiers that end with a “7”
 - ▶ All **mol_functions** that include “RNA” or “DNA”
 - ▶ All **modifications** that include “lysine”
 - ▶ All **biol_processes** that include “ATP”, “GTP” or “UTP”
- Optional:
 - ▶ Which protein whose names start with “H” has the biggest Mass?
 - ▶ Proteins whose names start with a “C” and end with a “1”

- The IN operator allows you to specify multiple values in a WHERE clause
- The IN operator is a shorthand for multiple OR conditions
- You can use it by writing multiple values inside brackets after the statement IN and as soon as a row contains one of these values it is picked by SELECT
- Syntax:

```
SELECT column_name(s)
```

```
FROM table_name
```

```
WHERE column_name IN (value1, value2, ...);
```


- We can also use IN together with a so called sub query, that means a second SELECT statement in our condition:

```
SELECT column_name(s)
```

```
FROM table_name
```

```
WHERE column_name IN (SELECT STATEMENT);
```

- When using such sub queries SQL first executes the sub query and produces the corresponding result set
- This result set is then used to filter the rows of the outer SELECT statement

ID	Name	Taxonomy	King_ID
1	Human	homo sapiens sapiens	7
2	C. subterraneum	Caldiarchaeum subterraneum	2
3	Mouse	mus musculus	7
4	E. coli	Escherichia coli	1

ID	Name
1	Bacteria
2	Archaea
3	Protozoa
4	Chromista
5	Plantae
6	Fungi
7	Animalia
8	Viruses

- Imagine the two tables (organisms at the top, kingdoms on the left) you can see here and now we want to use IN to get information about the kingdoms we have organisms for
- To do this we need to create a SELECT statement including a subquery to see which values we can find in the column King_ID (a foreign key referencing ID in the table kingdoms)

ID	Name	Taxonomy	King_ID
1	Human	homo sapiens sapiens	7
2	C. subterraneum	Caldiarchaeum subterraneum	2
3	Mouse	mus musculus	7
4	E. coli	Escherichia coli	1

ID	Name
1	Bacteria
2	Archaea
3	Protozoa
4	Chromista
5	Plantae
6	Fungi
7	Animalia
8	Viruses

- This subquery is relative simple:

```
SELECT King_ID FROM organisms;
```

- The result set will contain the values 1, 2 and 7 (twice)
- Now we can use this subquery in an outer SELECT statement with a WHERE clause which is able to compare the values of the result set with the values in the column ID of the table kingdoms

ID	Name	Taxonomy	King_ID
1	Human	homo sapiens sapiens	7
2	C. subterraneum	Caldiarchaeum subterraneum	2
3	Mouse	mus musculus	7
4	E. coli	Escherichia coli	1

ID	Name
1	Bacteria
2	Archaea
3	Protozoa
4	Chromista
5	Plantae
6	Fungi
7	Animalia
8	Viruses

- To do so we need a SELECT statement as follows:

```
SELECT * FROM kingdoms WHERE ID IN ...
```

- Now we have created both parts of our statement and only need to combine them:

```
SELECT * FROM kingdoms
```

```
WHERE ID IN (SELECT King_ID FROM organisms);
```

ID	Name	Taxonomy	King_ID
1	Human	homo sapiens sapiens	7
2	C. subterraneum	Caldiarchaeum subterraneum	2
3	Mouse	mus musculus	7
4	E. coli	Escherichia coli	1

ID	Name
1	Bacteria
2	Archaea
3	Protozoa
4	Chromista
5	Plantae
6	Fungi
7	Animalia
8	Viruses

- The result of this complex SELECT statement is now the small table we can see below:

ID	Name
1	Bacteria
2	Archaea
7	Animalia

```
SELECT * FROM proteins WHERE  
Organism_ID IN (1,5,7,22,30);
```

```
SELECT * FROM proteins  
WHERE Organism_ID NOT IN (1,5,7,22,30);
```

```
SELECT * FROM kingdoms  
WHERE ID IN (SELECT Kingdom_ID FROM organisms);
```

- Write SELECT statements that fulfill the following criteria:
 - ▶ All **proteins** that have Organism_IDs of 1, 3, 28, 21 or 22
 - ▶ All **organisms** that have Kingdom_IDs of 1, 2, 3, 4 or 8
 - ▶ For which **organisms** do we have proteins in our database
 - ▶ All **IUPAC Names** for the Hetero_IDs 1, 6, 7 or 14
 - ▶ All **proteins** for which structures with a Source_ID of 1 exist
- Optional:
 - ▶ Which proteins have structures with a Resolution?
 - ▶ Which proteins are in locations with descriptions?
 - ▶ Which proteins have modifications with a melting point?

- The BETWEEN operator selects values within a given range
- The values can be numbers, text or dates
- The BETWEEN operator is inclusive, that means begin and end values are included
- Syntax:

```
SELECT column_name(s)
```

```
FROM table_name
```

```
WHERE column_name BETWEEN startvalue AND endvalue;
```


- When working with BETWEEN keep in mind that the startvalue ALWAYS needs to be lower than the endvalue
- For numerical values this is self-explanatory
- For dates the “earlier” one is the smaller value, that means 01.01.2020 is smaller than 31.12.2022 for example
- For text values, the order is again alphabetically with “a” being the lowest value and “z” the highest value, so your values need to be in alphabetical order

```
SELECT * FROM proteins WHERE  
Protein_Length BETWEEN 100 AND 200;
```

```
SELECT * FROM proteins  
WHERE Protein_Length NOT BETWEEN 100 AND 200;
```

```
SELECT * FROM proteins  
WHERE Protein_Length BETWEEN 100 AND 200  
AND Organism_ID NOT IN (1,2,3);
```

```
SELECT * FROM proteins
WHERE Protein_Name BETWEEN 'Annexin A3' AND 'Histone H3.2'
ORDER BY Protein_Name;
```

```
SELECT * FROM proteins
WHERE Protein_Name NOT BETWEEN 'Annexin A3' AND 'Histone H3.2'
ORDER BY Protein_Name;
```

```
SELECT * FROM proteins
WHERE Protein_Name BETWEEN 'Transposase' AND 'Trypanothione reductase'
ORDER BY Organism_ID;
```

- Write SELECT statements that fulfill the following criteria:
 - ▶ All **proteins** whose mass is between 25000 and 50000
 - ▶ All **structural_data** whose resolution is between 2.1 and 2.8
 - ▶ All **organisms** between V. cholerae and S. flexneri
 - ▶ All **domains** between MG binding site and FE binding site
 - ▶ All **proteins** whose length is between 100 and 200 ordered by Name
- Optional:
 - ▶ proteins with a length between 300 and 600 where the mass is higher than 50000
 - ▶ structures with sidechain outliers between 0 and 1 with a resolution smaller than 2.0

- The LIMIT clause is used to specify the number of records to return
- The LIMIT clause is especially useful on large tables, since returning a large number of records can impact the performance of your statements
- Syntax:

SELECT column_name(s)

FROM table_name

WHERE condition

LIMIT number;

```
SELECT * FROM organisms LIMIT 3;
```

```
SELECT Organism_name, Taxonomy FROM organisms  
ORDER BY Kingdom_ID LIMIT 6;
```

```
SELECT * FROM organisms  
WHERE Kingdom_ID=1 LIMIT 3;
```

- Write SELECT statements that fulfill the following criteria:
 - ▶ The first ten **proteins**
 - ▶ The first four **organisms** that have a Kingdom_ID of 2
 - ▶ The first six **structures** with a Method_ID of 2
 - ▶ The first 12 **proteins** ordered by length
 - ▶ The first 5 **proteins** with Organism_ID of 1 ordered by descending Mass
- Optional:
 - ▶ The first 3 locations with a description and GO-Term higher than 10000
 - ▶ The first 4 proteins with Organism_ID of 1 ordered by Mass
 - ▶ The first 20 structures from AlphaFold (Source_ID) ordered by identifier

Aggregate Functions

- The MIN() function returns the smallest value of the selected column
- The MAX() function returns the largest value of the selected column
- In combination with numerical columns the values are selected according to their numerical values
- In combination with text columns the values are selected according to their alphabetical values, by seeing the letter “a” as lowest and the letter “z” as highest value

- MIN() Syntax:

```
SELECT MIN(column_name)
```

```
FROM table_name
```

```
WHERE condition;
```

- MAX() Syntax:

```
SELECT MAX(column_name)
```

```
FROM table_name
```

```
WHERE condition;
```

```
SELECT MIN(Mass) FROM proteins;
```

➔ Smallest Mass

```
SELECT MAX(Mass) FROM proteins;
```

➔ Highest Mass

```
SELECT MAX(Mass) FROM proteins  
WHERE Protein_Length<=500;
```

➔ Highest Mass for Protein_Length<=500

```
SELECT * FROM proteins  
WHERE Mass=(SELECT Min(Mass) FROM proteins);
```

➔ Complete Row of smallest Mass

```
SELECT MIN(Protein_Name) FROM proteins;
```

➔ First alphabetical entry

```
SELECT MAX(Protein_Name) FROM proteins;
```

➔ Last alphabetical entry

- Write SELECT statements that hand you back the following:
 - ▶ The biggest Protein_Length in **proteins**
 - ▶ The smallest Resolution in **structural_data**
 - ▶ The alphabetically first taxonomic name of an **organism**
 - ▶ The biggest Mass for **proteins** with less than 400 amino acids
 - ▶ The alphabetically last name of a **modification**
 - ▶ The complete row for the shortest **protein**
 - ▶ The complete row for the **protein** with the biggest Mass
- Optional:
 - ▶ Rows of domains with the lowest maximum repeats ordered by Annotation

- The COUNT() function returns the number of rows that match a specified criterion
- This can be mainly used to determine how many rows your result table will have
- Syntax:

```
SELECT COUNT(column_name)
FROM table_name
WHERE condition;
```

- Example:

```
SELECT COUNT(Organism_ID) FROM proteins;
```

- Using COUNT without any condition as shown before will tell you how many rows of your table have a value in the corresponding column
- One especially useful usage of COUNT is in combination with DISTINCT to determine how many different values a certain column has:

```
SELECT COUNT(DISTINCT Organism_ID) FROM proteins;
```

- The AVG() function returns the average value of all fields in a numeric column that have a value (empty fields are ignored)
- The result will always be a floating point number, even if your column contains only integer values

- Syntax:

```
SELECT AVG(column_name)
```

```
FROM table_name
```

```
WHERE condition;
```

- Example:

```
SELECT AVG(Mass) FROM proteins;
```

- The SUM() function returns the total sum of all fields in a numeric column that contain a value (empty fields are ignored)

- Syntax:

```
SELECT SUM(column_name)
```

```
FROM table_name
```

```
WHERE condition;
```

- Examples:

```
SELECT SUM(Mass) FROM proteins;
```

```
SELECT SUM(Mass) FROM proteins WHERE Protein_Length<250;
```


- Write SELECT statements that answer the following questions:
 - ▶ How many **proteins** have an Organism_ID of 1?
 - ▶ What is the average length of **proteins**?
 - ▶ How many **organisms** have a Kingdom_ID of 1?
 - ▶ What is the complete length of all **proteins**?
 - ▶ What is the average length of **proteins** with a mass smaller than 15000?
 - ▶ What is the average resolution of all **structural_data** that have an R-free value?
- Optional:
 - ▶ Proteins with a Mass higher than the average Mass
 - ▶ Proteins with a length smaller than the average Length of proteins with a mass > 15000

- The GROUP BY statement groups rows that have the same values into summary rows, like "find the number of customers in each country"
- The GROUP BY statement is often used with aggregate functions (COUNT(), MAX(), MIN(), SUM(), AVG()) to group the result-set by one or more columns
- Syntax:

SELECT column_name(s)

FROM table_name

WHERE condition

GROUP BY column_name(s);

ID	Name	Length	Mass	Organism_ID
1	Calmodulin	169	16811	29
2	Uncharacterized Protein	608	67222	30
3	cDNA FLJ61389	824	95088	1
4	Outer membrane lipoprotein slyB	155	15602	31
5	UDP-glucose-glycoprotein	1509	170912	32
6	Amyloid-beta precursor protein	770	86943	1

- For the table above we just want to know how many proteins we have for the different organisms (Organism_ID) in our database
- We can achieve this by combining the aggregate function COUNT with the GROUP BY statement

ID	Name	Length	Mass	Organism_ID
1	Calmodulin	169	16811	29
2	Uncharacterized Protein	608	67222	30
3	cDNA FLJ61389	824	95088	1
4	Outer membrane lipoprotein slyB	155	15602	31
5	UDP-glucose-glycoprotein	1509	170912	32
6	Amyloid-beta precursor protein	770	86943	1

- To do so we would try to create a statement using COUNT as follows:

```
SELECT COUNT(ID), Organism_ID FROM proteins;
```

- However we can't execute the statement above since SQL can't combine a single value (COUNT(ID)) with a column of values (Organism_ID) in the result set

ID	Name	Length	Mass	Organism_ID
1	Calmodulin	169	16811	29
2	Uncharacterized Protein	608	67222	30
3	cDNA FLJ61389	824	95088	1
4	Outer membrane lipoprotein slyB	155	15602	31
5	UDP-glucose-glycoprotein	1509	170912	32
6	Amyloid-beta precursor protein	770	86943	1

- Therefore we need to add a GROUP BY statement to use the selected column Organism_ID in our SELECT statement:

```
SELECT COUNT(ID), Organism_ID FROM proteins GROUP BY Organism_ID;
```

- Now we count the values in the column ID for each different value in the column Organism_ID

ID	Name	Length	Mass	Organism_ID
1	Calmodulin	169	16811	29
2	Uncharacterized Protein	608	67222	30
3	cDNA FLJ61389	824	95088	1
4	Outer membrane lipoprotein slyB	155	15602	31
5	UDP-glucose-glycoprotein	1509	170912	32
6	Amyloid-beta precursor protein	770	86943	1

COUNT(ID)	Organism_ID
2	1
1	29
1	30
1	31
1	32

- Since the value 1 appears twice in the column Organism_ID we get back a value of 2 for COUNT(ID) in this case
- The other values in the column Organism_ID only appear once each so the result for COUNT(ID) in these cases is 1

```
SELECT COUNT(ID), Organism_ID  
FROM proteins  
GROUP BY Organism_ID;
```

- Since COUNT(ID) will be a new column in our result table we can even sort according to the results of it:

```
SELECT COUNT(ID), Organism_ID  
FROM proteins  
GROUP BY Organism_ID  
ORDER BY COUNT(ID) DESC;
```

- Write SELECT statements that fulfill the following criteria:
 - ▶ Show the number of Protein_IDs for the different structure_IDs in **secondary_protein**
 - ▶ Show the number of **organisms** for the different Kingdom_IDs
 - ▶ Show the number of **structures** for the different Method_IDs
 - ▶ Show the number of Protein_IDs of the different Domain_IDs in **domains_proteins**
- Optional:
 - ▶ Overall mass for proteins from different organisms
 - ▶ Average length for proteins from different organisms with an annotation bigger than 3
 - ▶ Average resolution for different clashscores of structures with an R-free value
 - ▶ Overall weight of modifications grouped by the amount of hydrogen bond donors

- The HAVING clause was added to SQL because the WHERE keyword cannot be used with aggregate functions
- Syntax:

SELECT column_name(s)

FROM table_name

WHERE condition

GROUP BY column_name(s)

HAVING condition;

- We need to use HAVING as soon as we want to filter our result set according to values in a column that was created using an aggregate function, like COUNT for example
- We can add a HAVING statement to our statement we created earlier when talking about GROUP BY:

```
SELECT COUNT(ID), Organism_ID FROM proteins GROUP BY Organism_ID;
```

- As we saw this statement produces the result set we can see on the right side
- Now we add a HAVING statement to it to filter according to the values in the column COUNT(ID) of our result set:

```
SELECT COUNT(ID), Organism_ID FROM proteins  
GROUP BY Organism_ID HAVING COUNT(ID) > 1;
```

- Now we get back only the rows in our result set where the value in the column COUNT(ID) is bigger than 1

COUNT(ID)	Organism_ID
2	1
1	29
1	30
1	31
1	32

COUNT(ID)	Organism_ID
2	1

```
SELECT COUNT(ID), Organism_ID  
FROM proteins  
GROUP BY Organism_ID  
HAVING COUNT(ID) > 2;
```

```
SELECT COUNT(ID), Organism_ID  
FROM proteins  
GROUP BY Organism_ID  
HAVING COUNT(ID) > 2  
ORDER BY COUNT(ID) ASC;
```

- Write SELECT statements that answer the following questions:
 - ▶ Which domains appear at least 10 times in proteins (**domains_proteins**)?
 - ▶ Which molecular functions are executed by at least 2 different proteins (**protein_function**)?
 - ▶ Which kingdoms have more than one **organism** in the database?
 - ▶ Which modifications appear in at least 20 proteins (**modifications_protein**)?
- Optional:
 - ▶ Which organisms have proteins with an overall Mass bigger than 100000?
 - ▶ Which organisms have proteins with an average length smaller than 350?

Combining Tables

- A JOIN statement is used to combine rows from two or more tables, based on a related column between them
- These related columns are often a Foreign Key column and the referenced Primary Key column
- Example:

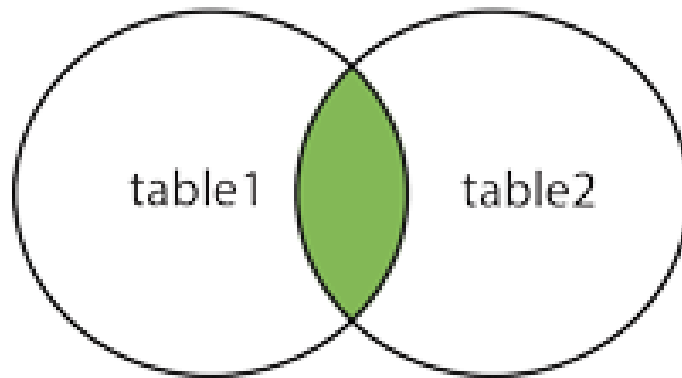
```
SELECT organisms.Taxonomy, organisms.Organism_name,  
proteins.Protein_Name, proteins.Protein_Length,  
FROM organisms INNER JOIN proteins  
ON organisms.Organism_ID=proteins.Organism_ID;
```

- Let's go through the example we saw one by one
- First we select columns from two tables:
 - ▶ `SELECT organisms.Taxonomy, organisms.Organism_name,`
`proteins.Protein_Name, proteins.Protein_Length,`
 - ▶ The point notation allows you to reference columns from multiple tables
 - ▶ The syntax is always `table.columnname`
- Second we specify the table we want to work with and how to JOIN them:
 - ▶ `FROM organisms INNER JOIN proteins`
 - ▶ Here organisms is the left table and proteins the right table
 - ▶ Furthermore we want to create an INNER JOIN

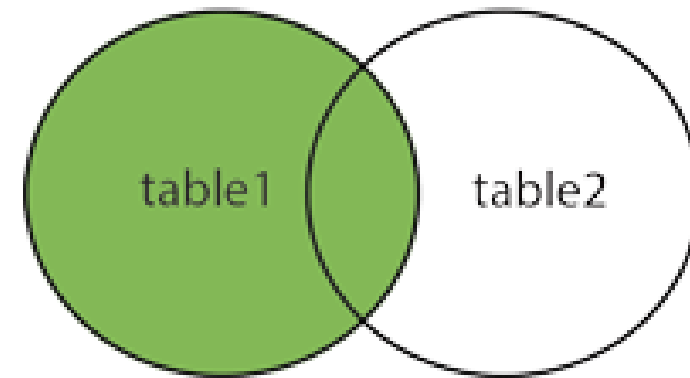
- Third we need to specify which columns should be used to create our JOIN:
 - ▶ `ON organisms.Organism_ID=proteins.Organism_ID`
 - ▶ Here I make use of the Foreign Key column `Organism_ID` in `proteins` and the referenced Primary Key column `Organism_ID` in `organisms`
 - ▶ The order is not important, however it makes the most sense to use the same order in which you wrote your tables

- (INNER) JOIN: Returns records that have matching values in both tables
- LEFT (OUTER) JOIN: Returns all records from the left table, and the matched records from the right table
- RIGHT (OUTER) JOIN: Returns all records from the right table, and the matched records from the left table
- FULL (OUTER) JOIN: Returns all records when there is a match in either left or right table → NOT supported by MySQL

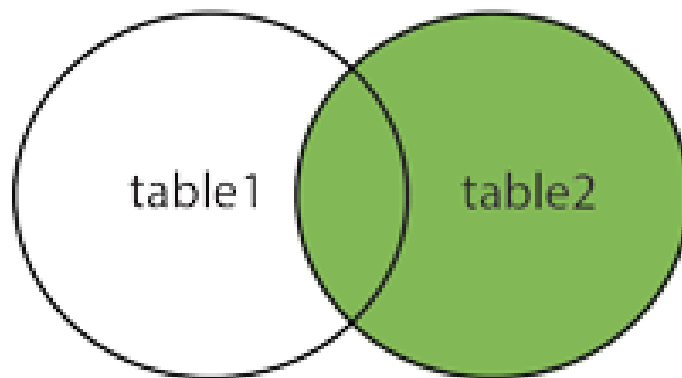
INNER JOIN



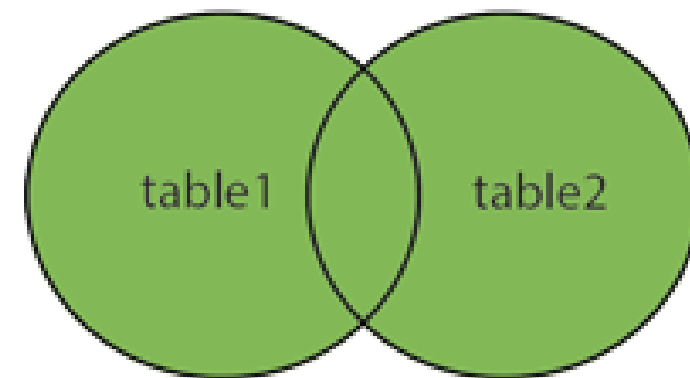
LEFT JOIN



RIGHT JOIN



FULL OUTER JOIN



- The INNER JOIN keyword selects records that have matching values in both tables.
- Syntax:

```
SELECT column_name(s)
```

```
FROM table1
```

```
INNER JOIN table2
```

```
ON table1.column_name = table2.column_name;
```

```
SELECT organisms.Taxonomy, organisms.Organism_name,  
proteins.Protein_Name, proteins.Protein_Length,  
FROM organisms INNER JOIN proteins  
ON organisms.Organism_ID=proteins.Organism_ID;
```

- The LEFT JOIN keyword returns all records from the left table (table1), and the matching records from the right table (table2)
- If there is no matching data in the right table you will get NULL records
- Syntax:

```
SELECT column_name(s)
```

```
FROM table1
```

```
LEFT JOIN table2
```

```
ON table1.column_name = table2.column_name;
```

```
SELECT organisms.Taxonomy, organisms.Organism_name,  
proteins.Protein_Name, proteins.Protein_Length  
FROM organisms LEFT JOIN proteins  
ON organisms.Organism_ID=proteins.Organism_ID;
```

- The RIGHT JOIN keyword returns all records from the right table (table2), and the matching records from the left table (table1)
- If there is no matching data in the left table you will get NULL records
- Syntax:

```
SELECT column_name(s)
```

```
FROM table1
```

```
RIGHT JOIN table2
```

```
ON table1.column_name = table2.column_name;
```



```
SELECT organisms.Taxonomy, organisms.Organism_name,  
proteins.Protein_Name, proteins.Protein_Length  
FROM organisms Right JOIN proteins  
ON organisms.Organism_ID=proteins.Organism_ID;
```

- Write SELECT statements making use of the JOIN statement for the following tasks:
 - ▶ Combine **Organism** Names and **Kingdom** Names
 - ▶ Combine **Protein** Names, Lengths with **Structure** Identifiers and total lengths
 - ▶ Combine Ligand_Name (**hetero_atoms**) with **atom_information** about charge and mass
 - ▶ Combine the names of **modifications** with **modification_data** about weight, CAS- and EC-Number
- Optional:
 - ▶ Combine Organism Name and Protein Name and Mass for all proteins longer than 250
 - ▶ Combine Taxonomic Names and Kingdom names for Organism from the Kingdoms 1,2 & 3

- The UNION operator is used to combine the result-set of two or more SELECT statements:
 - ▶ Every SELECT statement within UNION must have the same number of columns
 - ▶ The columns must also have similar data types
 - ▶ The columns in every SELECT statement must also be in the same order
- Syntax:

```
SELECT column_name(s) FROM table1
```

```
UNION
```

```
SELECT column_name(s) FROM table2;
```

- The UNION statement allows you to combine the results of two SELECT statements while putting the result on top of each other
- This is especially useful, when you have tables with very similar columns, for example a customer and staff table and you want to create a mailing list:

```
SELECT first_name, last_name, email_address FROM customers
```

```
UNION
```

```
SELECT first_name, last_name, email_address FROM staff;
```

- In our database we could use UNION to get a list of all IUPAC names saved in `modification_data` and `iupac_names`:

```
SELECT IUPAC_Name FROM modification_data  
  
UNION  
  
SELECT IUPAC_Name FROM iupac_names;
```

- The UNION statement removes duplicates when combining the two SELECT statements
- Especially when dealing with large databases this can influence the performance of the statement negatively
- To avoid this you can use UNION ALL which simply places all rows from both SELECT statements into the result set:

```
SELECT Protein_ID FROM protein_function  
  
UNION ALL  
  
SELECT Protein_ID FROM protein_location;
```

- As long as the two SELECT statements produce result sets that have the same amount and format of columns you can use everything you learned so far
- That means you can include conditions using WHERE as well:

```
SELECT IUPAC_Name FROM modification_data  
  
WHERE IUPAC_Name LIKE "%phospho%"  
  
UNION ALL  
  
SELECT IUPAC_Name FROM iupac_names  
  
WHERE IUPAC_Name LIKE "%phospho%";
```

- Write SQL statements that combine the following columns from different tables using UNION:
 - ▶ Names of **biol_processes** and **mol_functions**
 - ▶ The mass of **atom_information** and **modification_data**
 - ▶ The names of **sources** and **methods**
- Optional:
 - ▶ All Protein_IDs from protein_function, protein_location, protein_process and domains_proteins

Views

- As we learned the result tables our SELECT statements produce are only temporary but sometimes we want to work with these result tables even more
- In these cases we can create a VIEW, a table based on the result-set of an SQL statement
- It contains rows and columns, just like a real table
- The fields in a view are fields from one or more real tables in the database
- You can add SQL statements and functions to a view and present the data as if the data was from one single table
- A view is created with the CREATE VIEW statement in combination with a SELECT statement

- A view is an object that is added to our database and therefore accessible to us via the sidebar, similar to our tables
- Views are always connected to the tables they were created from, that means if we enter new data into a table that was used to create a view, this data will afterwards also appear in the corresponding view
- The same is true for the reverse process, that means adding data to a view, in this case the new data will also appear in the corresponding table

```
CREATE VIEW view_name AS  
SELECT column_name(s)  
FROM table_name  
WHERE condition;
```

IMPORTANT: You can't create views if they're already existing

```
CREATE VIEW HumanProteins AS  
SELECT * FROM proteins WHERE Organism_ID = 1;
```

```
SELECT * FROM HumanProteins WHERE  
Annotation=1;
```

```
CREATE VIEW BiggerThanAverage AS  
SELECT * FROM proteins WHERE Protein_Length >  
(SELECT AVG(Protein_Length) FROM proteins);
```

- Create Views that contain the following data:
 - ▶ All **proteins** that are longer than the average length
 - ▶ All **structural_data** that have a resolution smaller than the average resolution
 - ▶ All **proteins** that have an Organism_ID of 1 and a mass bigger than the average mass
 - ▶ All **modification_data** with a molecular weight bigger than 200 and a Melting _Point
- Optional:
 - ▶ All structures that have a resolution smaller than the average resolution
 - ▶ All proteins that have more than one Domain_ID in domains_proteins

- A view can be updated with the CREATE OR REPLACE VIEW statement
- This can be used for example to adjust the condition we specified when creating our view
- We can also change which columns are presented in our view with the help of this statement
- Syntax:

```
CREATE OR REPLACE VIEW view_name AS
```

```
SELECT column_name(s)
```

```
FROM table_name
```

```
WHERE condition;
```

- Let's say I want to use only two columns instead of all columns in the VIEW we created earlier
- In this case we can use the CREATE OR REPLACE VIEW statement and adjust the picked columns as follows:

```
CREATE OR REPLACE VIEW BiggerThanAverage AS  
  
SELECT Protein_Length, Protein_Name FROM proteins WHERE  
  
Protein_Length > (SELECT AVG(Protein_Length) FROM proteins);
```

- Adjust the Views from Exercise 17 so that they contain the following data:
 - ▶ Names and Lengths of **proteins** that are longer than the average length
 - ▶ Identifier, R_free and Resolution of **structural_data** that have a resolution smaller than the average resolution
 - ▶ All **proteins** that have an Organism_ID of 1 or 2 and a mass bigger than the average mass
 - ▶ All **modification_data** with a molecular weight bigger than 150 and a Melting_Point
- Optional:
 - ▶ Identifier, Source_ID and Method_ID of structures that have a resolution smaller than the average resolution
 - ▶ Names and Lengths of proteins that have more than one Domain_ID in domains_proteins

Stored Procedures

- A stored procedure is a prepared SQL code that you can save
- The code can then be reused over and over again
- So if you have an SQL query that you write over and over again, save it as a stored procedure
- Afterwards just call it to execute it
- You can also pass parameters to a stored procedure, so that the stored procedure can act based on the parameter value(s) that is passed similar to functions in Python

- Different SQL versions have different syntax for creating stored procedures
- Here we will discuss the MySQL version using the statements BEGIN, END and CALL
- Other SQL versions use the statements AS, GO and EXEC
- Similar to Views you can only create Procedures if they don't exist already

- The general syntax for creating a stored procedure in MySQL is as follows:

```
DELIMITER //
```

```
CREATE PROCEDURE procedure_name ()
```

```
BEGIN
```

```
    SQL statements
```

```
END //
```

```
DELIMITER ;
```

- The Statement DELIMITER is used to switch between different line-endings
MySQL recognizes (default is “;”)
- The inclusion of this statement is needed to be able to add a semi-colon “;” to the SELECT statement inside our procedure
- After the procedure was created we switch back to the semi-colon as line-ending by using DELIMITER again

- In between the two DELIMITOR statements we have the CREATE PROCEDURE statement that allows you to create a stored procedure
- A procedure that allows us to select all rows from the table proteins could then look as follows:

```
CREATE PROCEDURE SelectAllProteins ()  
  
BEGIN  
  
    SELECT * FROM Proteins;  
  
END //
```

- In the code above SelectAllProteins is the name of the procedure and SELECT * FROM Proteins; is the statement that is executed everytime we call it later

- To execute our procedure we then need the CALL statement:

```
CALL SelectAllProteins();
```

- As you can see you need to add empty brackets after the name of the procedure to be able to execute it properly

- Create stored procedures for the following tasks:
 - ▶ Select all columns from **organism**
 - ▶ Select all columns from **structures** where the Source_ID is 2
 - ▶ Select Identifier, Resolution and Release Date from **structural_data**
 - ▶ Select all rows from **secondary_structure** where the name is Helix
 - ▶ Select Protein_Name and Protein_Length for **Proteins** longer than 250
- Optional:
 - ▶ Select all rows of kingdoms for which organisms are in the database
 - ▶ Select all rows of proteins for which structures with a source_ID of 1 are in the database

- Creating stored procedures for statements we want to use over and over again might not seem very useful, since we could achieve something similar with simple scripts as well
- However stored procedures allow you to hand over parameters to them and produce results according to these values

- To make use of this feature we have to use the statement IN during the creation of the procedure:

```
CREATE PROCEDURE ProteinOrganisms (IN IDinput int)

BEGIN

    SELECT * FROM Proteins WHERE Organism_ID=IDinput;

END //
```

- In the statement above I specify the parameter IDinput as integer (int) and can then use it inside my procedure to filter accordingly
- You always need to specify the datatype of the parameter you want to hand over

- Once you defined a procedure with a parameter you can execute it similar to before by using the CALL statement:

```
CALL ProteinOrganims (1);
```

- Now however you need to write the value for the parameter inside the brackets, here 1

- You can even define procedures that use multiple parameters by adding multiple parameter definitions inside the brackets:

```
CREATE PROCEDURE ProteinSize (IN MassInput int, LengthInput int)
BEGIN
    SELECT * FROM Proteins WHERE
        Mass > MassInput AND Protein_Length > LengthInput;
END //
```

- By simply comma separating them I was able to create a procedure with 2 parameters

- Now you can execute your procedure with the CALL statement again
- This time however you need to hand over two values comma separated so that both parameters can be used:

```
CALL ProteinSize(250000,300);
```

- Create stored procedures for the following tasks:
 - ▶ **Organisms** for a specific Kingdom_ID
 - ▶ **Structural_data** with a Resolution higher than a specific value
 - ▶ **Atom_information** with a mass smaller than a specific value
 - ▶ **Modification_data** with specific values for hydrogen bond donors and acceptors
 - ▶ **Secondary_structures** with a specific name
- Optional:
 - ▶ Proteins whose names contain a specific pattern and are longer than another specific value
 - ▶ Proteins having structures with a specific resolution and a specific R-free value

Multiple databases

- SQL Servers can host more than just one database
- That's the reason why we need the USE statement to switch between different databases
- If you don't want to always switch between different databases or if you want to work on tables from different databases at once SQL allows you to do so as well
- In these cases you need a point notation to tell SQL which table from which database you mean
- The general syntax is:
 - `databaseName.tableName`

- For example you can execute SELECT statements on tables from different databases in one script as follows:

```
SELECT * FROM databasecq.proteins;
```

```
SELECT * FROM magicexample.decks;
```

- As you can see I added the database name (databasecq or magicexample) before the table name (proteins or decks)
- Like this it doesn't matter which database is active you have access to all tables on the whole server

Creating Databases

- When working with SQL the normal approach is to generate the database first
- During this process you create the different tables and of course you need to include things like Primary Keys and Foreign Keys at this time as well
- Therefore it's essential to have a good understanding about the database you want to create and which tables are related to each other
- Always keep in mind to create the parent tables before the child tables, to make sure that your Foreign Keys can work as you need them to
- On the next slide I will give you a brief overview about the steps when creating a database in SQL

1. Collect information about the database and model all tables you will need theoretically, including datatypes and constraints (more about this later)
2. Analyze the relations between your tables to understand the hierarchy between them
3. Create the tables in SQL by starting with parent tables and moving through the hierarchy you developed in step 2. Don't forget to include Foreign Keys in this step to relate your tables with each other
4. Fill your tables with data, again start with parent tables and move through your hierarchy of step 2

- In SQL we can create a database with a very simple statement called CREATE DATABASE

- Syntax:

`CREATE DATABASE databasename;`

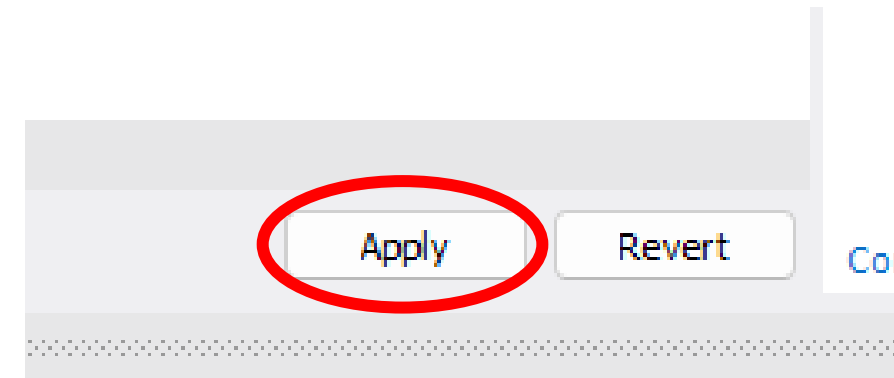
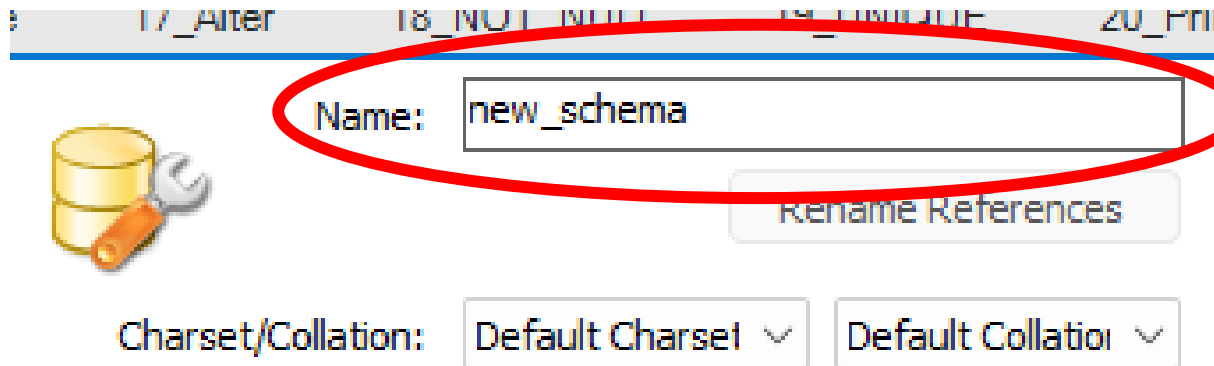
- For example I could create a database called magic as follows:

`CREATE DATABASE magic;`

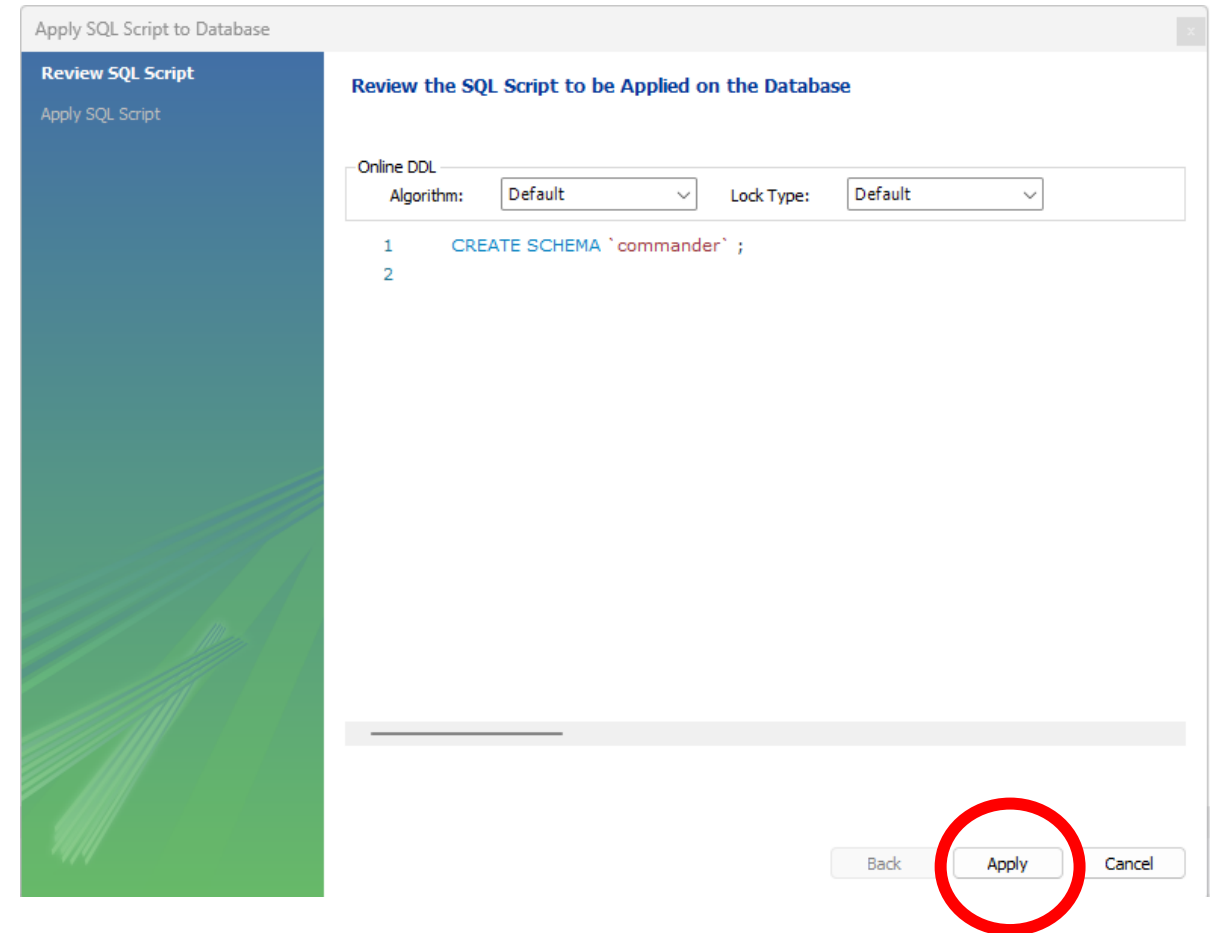
- When creating databases we can do so by using SQL statements as well as with the help of the GUI
- For creating databases we have the button “Create a new schema in the connected server” (see image below) which opens a new tab (next slide)



- Once you clicked on the button, a new tab will open where you can specify the name of your new database (left image)
- After you typed in the name of your new database, you need to click on the button Apply on the bottom right of the tab (right image)



- After you clicked on Apply a Pop-Up, where you can see the SQL statements needed for creating your database / schema
- After you checked everything click on Apply again to execute your statements
- After the statements were executed, click on Finish to end the process



- In SQL we need to select / use a database to be able to run our statements on them

- To do this we can take the statement USE and the following syntax:

`USE databasename;`

- So to work on our newly created database magic it would look as follows:

`USE magic;`

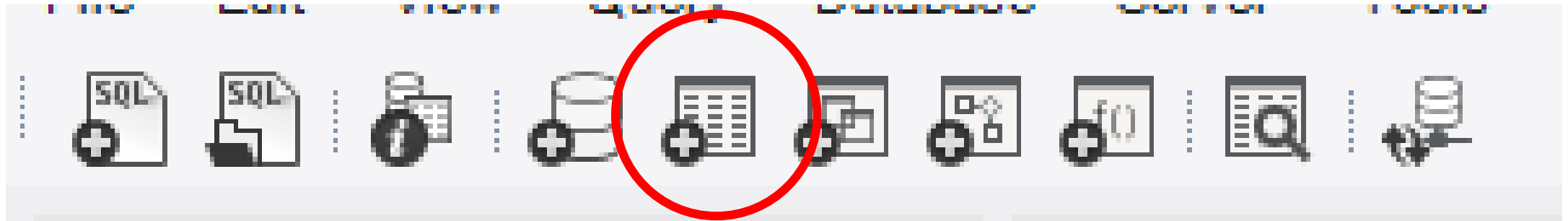
- Instead of using the statement USE, you can always double click on the name of the database (in the side bar) to activate it as I mentioned in the beginning

- The CREATE TABLE statement is used to create a new table in a database
- You always have to specify the datatype of your column, e.g. integer or float, and only values that are of the specified datatype can later be entered into your table
- Syntax:

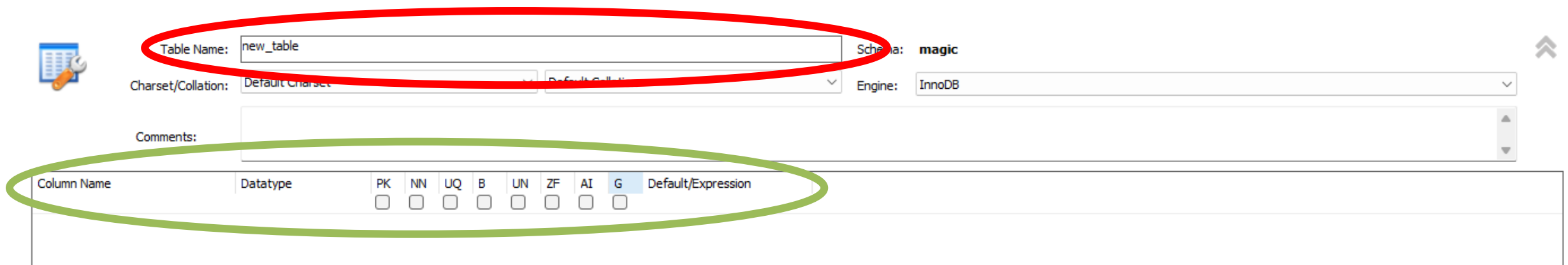
```
CREATE TABLE table_name (  
    column1 datatype,  
    column2 datatype,  
    column3 datatype,  
    ....);
```

```
CREATE TABLE decks (  
    ID int,  
    Short varchar(255),  
    Colors varchar(255),  
    FullName varchar(255),  
    Turn int,  
    Price float,  
    Box_ID int  
);
```

- Again instead of using a SQL statement you can use the GUI
- For creating tables we can use the button “Create a new table in the active schema in connected server” (red circle)



- After you clicked on this button a new tab open were you can specify the name of your table (red circle)
- Afterwards you can define the needed columns by specifying the name and datatype one by one (green circle)



The screenshot shows a database creation interface. A red circle highlights the 'Table Name' field, which contains 'new_table'. A green circle highlights the column definition table below the 'Comments' field. The table has columns for 'Column Name', 'Datatype', 'PK', 'NN', 'UQ', 'B', 'UN', 'ZF', 'AI', 'G', and 'Default/Expression'. The 'G' checkbox is selected.

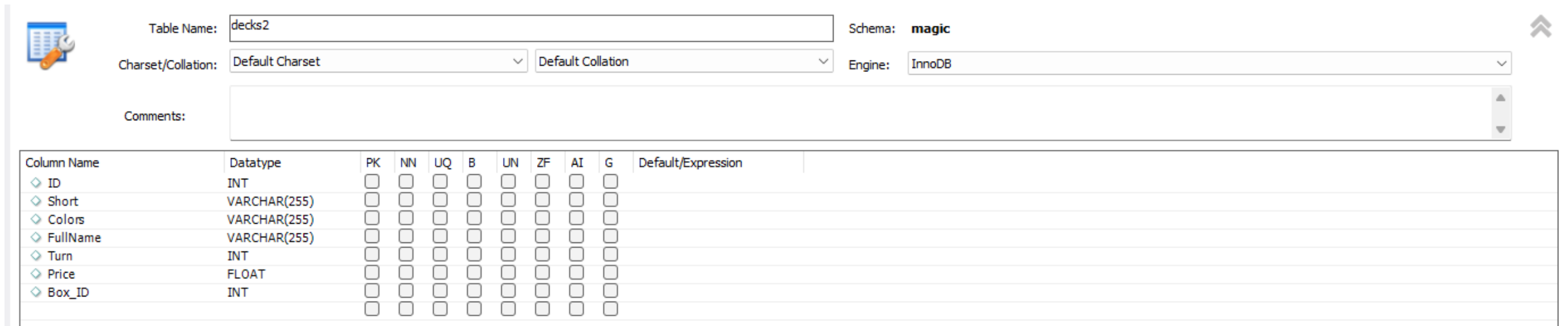
Table Name: Schema: **magic**

Charset/Collation: Engine:

Comments:

Column Name	Datatype	PK	NN	UQ	B	UN	ZF	AI	G	Default/Expression
		☐	☐	☐	☐	☐	☐	☐	☒	

- After you define all columns the tab should look similar to what you can see in the image below
- Afterwards you need to again click on Apply and go on just as we did for creating a database



The screenshot shows the 'CREATE TABLE' dialog in MySQL Workbench. The table name is 'decks2', the schema is 'magic', and the engine is 'InnoDB'. The charset and collation are set to 'Default'. The table has seven columns: ID (INT), Short (VARCHAR(255)), Colors (VARCHAR(255)), FullName (VARCHAR(255)), Turn (INT), Price (FLOAT), and Box_ID (INT). Each column has a 'PK' checkbox, and there are checkboxes for 'NN', 'UQ', 'B', 'UN', 'ZF', 'AI', and 'G'.

Column Name	Datatype	PK	NN	UQ	B	UN	ZF	AI	G	Default/Expression
ID	INT	☐	☐	☐	☐	☐	☐	☐	☐	
Short	VARCHAR(255)	☐	☐	☐	☐	☐	☐	☐	☐	
Colors	VARCHAR(255)	☐	☐	☐	☐	☐	☐	☐	☐	
FullName	VARCHAR(255)	☐	☐	☐	☐	☐	☐	☐	☐	
Turn	INT	☐	☐	☐	☐	☐	☐	☐	☐	
Price	FLOAT	☐	☐	☐	☐	☐	☐	☐	☐	
Box_ID	INT	☐	☐	☐	☐	☐	☐	☐	☐	

- When working with MySQL there are many datatypes available to us, which we can use when creating our tables
- The table down below summarizes some of the most common datatypes

Datatype	Description
char(n)	A string of the fixed length n
varchar(n)	A string of maximum length n
int	Integer values from -2147483648 up to 2147483647
bigint	Integer values from -9223372036854775808 up to 9223372036854775807
float	floating point numbers
decimal(n,d)	floating point number with a maximum of n digits and d digits after the decimal point
date	dates in format YYYY-MM-DD (2023-04-28)
year	year in four-digit format (2023)

- Create the following tables in a new database with the help of SQL statements. The column names are in brackets after the Table name:
 - ▶ books (ID, Book_Title, ISBN, Publication_Date, Publisher_ID, Page_Count, Price, Translator_ID, Original_Language)
 - ▶ publishers (ID, Publisher_Name, City, Address, EMail, Telephone)
 - ▶ authors (ID, Last_Name, Given_Name, Title, Birthdate, Birthplace)
 - ▶ wrote (Book_ID, Author_ID)
 - ▶ courses (ID, Course_Name, Course_Length, Room)
 - ▶ teachers (ID, Last_Name, Given_Name, Room, EMail, Telephone)
 - ▶ students (ID, Last_Name, Given_Name, EMail)
 - ▶ teaches (Course_ID, Teacher_ID, Weekday, Hours)
 - ▶ visits (Course_ID, Student_ID, Weekday, Hours)

- The TRUNCATE TABLE statement is used to delete the data inside a table, but not the table itself
- Syntax:
 - `TRUNCATE TABLE table_name;`
- Example:
 - `TRUNCATE TABLE decks2;`
- The same option is available to you in the right-click menu of the tables in the sidebar where we can see all our tables and databases

- The DROP TABLE statement is used to delete an existing table in a database
- Syntax:
 - `DROP TABLE table_name;`
- Example:
 - `DROP TABLE decks2;`
- Be careful before dropping a table because deleting a table will result in loss of complete information stored in the table!
- Again you are able to use the same function in the right-click menu of our sidebar

- Sometimes we need to make changes to our tables after we created them
- In these cases we can use the ALTER TABLE statement that allows us to:
 - add or remove columns
 - rename columns
 - rename tables
 - add or change constraints (more about this later)
- ALTER TABLE works always in combination with other statements such as ADD, RENAME, CHANGE, MODIFY or DROP
- All commands we discuss here are available to you in the right click menu of the corresponding table under “Alter Table...”

- We can use ALTER TABLE to add columns to our tables as shown down below
- Don't forget you also need to specify the datatype for these new columns as well
- Syntax:

```
ALTER TABLE table_name
```

```
ADD COLUMN column_name datatype;
```

- Examples:

```
ALTER TABLE decks ADD COLUMN Basics int;
```

```
ALTER TABLE decks ADD COLUMN NonBasics int;
```

```
ALTER TABLE decks ADD COLUMN Test varchar(100);
```

- To remove a column from a table, including all data saved in it, we can use the command DROP COLUMN with the name of the corresponding column afterwards
- Be careful when dropping things in SQL because the changes are permanent
- Syntax:

```
ALTER TABLE table_name
```

```
DROP COLUMN column_name;
```

- Example:

```
ALTER TABLE decks DROP COLUMN Test;
```

- To change the name of a table we can use the statement RENAME that we only can use for changing the names of tables
- To change the names of columns we need a different command (next slide)
- Syntax:

```
ALTER TABLE table_name  
RENAME new_name;
```

- Example:

```
ALTER TABLE decks RENAME decks3;
```

- You can use RENAME COLUMN to change the name of a column instead of the name of the table
- Syntax:

```
ALTER TABLE table_name
```

```
RENAME COLUMN old_column_name TO new_column_name;
```

- Example:

```
ALTER TABLE decks3
```

```
RENAME Short TO Shorts;
```

- To adjust the definition of a column you can use MODIFY
- Syntax:

```
ALTER TABLE table_name
```

```
MODIFY column_name datatype;
```

- Example:

```
ALTER TABLE decks3
```

```
MODIFY Shorts varchar(255);
```


- On the previous slides you saw how to make singular changes to your tables
- ALTER TABLE however lets you execute multiple changes on one table at once by simple comma separating the different adjustments:

```
ALTER TABLE boxes2  
  
ADD COLUMN Price float,  
  
ADD COLUMN Manufactor varchar(255),  
  
DROP COLUMN manufacturer,  
  
RENAME boxes,  
  
MODIFY Type varchar(50);
```

- In the example below I use all four operations on the table boxes2 at once

- Use SQL Statements to make the following changes to your existing tables:
 - ▶ Add the columns Title, Address and City to teachers
 - ▶ Add the column Postal_Code to publishers
 - ▶ Rename the Column Hours in teaches to Time_taught
 - ▶ Rename the Column Hours in visits to Time_visited
 - ▶ Rename the table wrote to written_by
 - ▶ Remove the column Translator_ID from the table books

- In SQL we cannot create tables that we already created earlier and we can also not delete things we didn't create beforehand
- In these cases we will get an error message and our statement is not executed correctly
- If you have a script with more than one command, the remaining script wouldn't be executed after you encountered an error, similar to what we learned when talking about Python and bash scripting

- You can use the if exists or if not exists statement to make sure that nothing is created twice or that your command doesn't produce an error than you run it on a (non-)existing table
- Syntax:
 - ▶ `DROP TABLE if exists table_name;`
 - ▶ `CREATE TABLE if NOT exists table_name (column1 datatype, column2 datatype, ...);`

Constraints

- SQL constraints are used to specify rules for the data in a table and are used to limit the type of data that can go into a table
- This ensures the accuracy and reliability of the data in the table
- If there is any violation between the constraint and the data action, the action is aborted
- Constraints can be column level or table level
 - Column level constraints apply to a column
 - Table level constraints apply to the whole table

- NOT NULL - Ensures that a column cannot have a NULL value, that means fields can't be empty
- UNIQUE - Ensures that all values in a column are different
- PRIMARY KEY - Uniquely identifies each row in a table
- FOREIGN KEY – Is used to model relations between tables by referencing an existing primary key
- CHECK - Ensures that the values in a column satisfies a specific condition
- DEFAULT - Sets a default value for a column if no value is specified when entering data

- Constraints can be specified when the table is created with the CREATE TABLE statement, or after the table is created with the ALTER TABLE statement
- The general syntax for these constraints differs from constraint to constraint, so we will go over them one by one
- You will see how to use each constraint separately but in reality you use many of them at the same time during the creation of your table, to include all needed rules directly in one statement
- Altering the tables by adding / removing the constraints should be done rarely

- By default, a column can hold NULL values, that means some fields can be empty
- The NOT NULL constraint enforces a column to NOT accept NULL values
- This enforces a field to always contain a value, which means that you cannot insert a new record, or update a record without adding a value to this field
- We can add this constraint to a column by simply writing the statement NOT NULL after the datatype of a column

- To add the NOT NULL constraint to a column simply write NOT NULL after the datatype of it in the CREATE TABLE statement:

```
CREATE TABLE boxes (  
    ID int NOT NULL,  
    Type varchar(255),  
    Subtype varchar(255),  
    manufacturer varchar(255)  
);
```

- The UNIQUE constraint ensures that all values in a column are different, as long as you have a value
- UNIQUE does not mean that there needs to be a value in this columns, just that if you enter a value it needs to be different from the remaining values
- Both the UNIQUE and PRIMARY KEY constraints provide a guarantee for uniqueness for a column or set of columns
- A PRIMARY KEY constraint automatically has a UNIQUE constraint
- However, you can have many UNIQUE constraints per table, but only one PRIMARY KEY constraint per table

- We can add the UNIQUE constraint in the same way we did with NOT NULL by simply adding it after the datatype during the table creation:

```
CREATE TABLE boxes (  
    ID int UNIQUE,  
    Type varchar(255),  
    Subtype varchar(255),  
    manufacturer varchar(255));
```

- Another way of adding the UNIQUE constraint is by adding the statement UNIQUE after the definition of all columns in the CREATE TABLE statement:

```
CREATE TABLE boxes (  
    ID int NOT NULL,  
    Type varchar(255),  
    Subtype varchar(255),  
    manufacturer varchar(255),  
    UNIQUE (ID));
```

- The advantage of using UNIQUE like this is, that we can combine multiple columns so that the combination of them needs to be unique

```
CREATE TABLE boxes (  
    ID int NOT NULL,  
    Type varchar(255),  
    Subtype varchar(255),  
    manufacturer varchar(255),  
    UNIQUE (ID, Type));
```

- The PRIMARY KEY constraint uniquely identifies each record in a table
- Primary keys must contain UNIQUE values, and cannot contain NULL values
- A table can have only ONE primary key
- In the table, this primary key can consist of single or multiple columns (fields)
- We can use the same syntax we used in combination with UNIQUE for PRIMARY KEY as well

```
CREATE TABLE boxes (  
    ID int PRIMARY KEY,  
    Type varchar(255),  
    Subtype varchar(255),  
    manufacturer varchar(255)  
);
```

- We can simply add this constraint to a column by writing the constraint name after the datatype of the corresponding column


```
CREATE TABLE boxes (  
    ID int NOT NULL,  
    Type varchar(255),  
    Subtype varchar(255),  
    manufacturer varchar(255),  
    PRIMARY KEY (ID)  
);
```

- Again we can add a PRIMARY KEY to the end of our CREATE TABLE statement after we defined all columns of our table

```
CREATE TABLE boxes (  
    ID int NOT NULL,  
    Type varchar(255),  
    Subtype varchar(255),  
    manufacturer varchar(255),  
    PRIMARY KEY (ID, Type)  
);
```

- Just like with UNIQUE we can use multiple columns in the brackets of the PRIMARY KEY constraint to combine them in this constraint
- This is how we can create composite primary keys, which sometimes can be useful, especially when working with junction tables for example

- Auto-increment allows a unique number to be generated automatically when a new record is inserted into a table
- Often this is the primary key field that we would like to be created automatically every time a new record is inserted
- In MySQL it only works with Primary Keys
- To add this constraint we use the method we already learned for NOT NULL by simply adding the words `AUTO_INCREMENT` after the datatype of our column definition

```
CREATE TABLE boxes (  
    ID int NOT NULL Auto_Increment,  
    Type varchar(255),  
    Subtype varchar(255),  
    manufacturer varchar(255),  
    PRIMARY KEY (ID)  
);
```

- MySQL uses the `AUTO_INCREMENT` keyword to increase the value of automatically entered values into the corresponding column
- By default, the starting value for `AUTO_INCREMENT` is 1, and it will increase by 1 for each new record
- To let the `AUTO_INCREMENT` sequence start with another value, use the following SQL statement:
 - ▶ `ALTER TABLE boxes7 AUTO_INCREMENT=100;`

- The FOREIGN KEY constraint is used to prevent actions that would destroy links between tables
- A FOREIGN KEY is a field (or collection of fields) in one table, that refers to the PRIMARY KEY in another table
- The table with the foreign key is called the child table, and the table with the primary key is called the referenced or parent table
- We introduce foreign keys in SQL to model our relations as we learned when talking about relational databases in general

- When doing so we can only create a Foreign Key when the referenced Primary Key already exists in our database, that means we need to create the parent table before the child table
- As soon as you created a column with the FOREIGN KEY constraint you can only add values to this column that are also present in the referenced PRIMARY KEY column
 - ▶ That means we need to fill the parent table before the child table
 - ▶ If the Primary Key column has the value 1, 2 and 3 we can only add these three values or nothing to the Foreign Key column that references the Primary Key column

- The general syntax for creating a foreign key is as follows:

FOREIGN KEY (curr_column(s)) REFERENCES ref_table(ref_column(s))

- The curr_column(s) are part of the table that carries the foreign key, that means the child table
- The ref_table is the table that contains the primary key we want to reference, that means this is our parent table
- The ref_column(s) are the columns which contain the primary key we want to reference
- When working with composite primary key the curr_columns need to match the order and datatypes of the ref_columns


```
CREATE TABLE decks (  
    ID int NOT NULL,  
    Short varchar(255) NOT NULL,  
    Colors varchar(255),  
    FullName varchar(255),  
    Turn int,  
    Price float,  
    Box_ID int NOT NULL,  
    PRIMARY KEY (ID),  
    FOREIGN KEY (Box_ID) REFERENCES boxes (ID)  
);
```

- Creating a foreign key is done by adding an additional line to the create table statement in which we define our foreign key and which column(s) it should reference

- The CHECK constraint is used to limit the value range that can be placed in a column
- If you define a CHECK constraint on a column it will allow only certain values for this column
- The CHECK constraint always needs to be added to the table after you defined all columns
- This can be done either in the CREATE TABLE statement or with the help of the ALTER TABLE statement, similar to how we would add UNIQUE for example (next slide)

- You can use mathematical checks like:
 - `Age >= 18`
- You can use equality checks like:
 - `City = 'sandnes'`
- You can use checks to see if a value is in a list:
 - `lastname in ('curie','newton','einstein','hawking')`
- You can combine checks with logical operators:
 - `Age >= 18 and Age < 65`
 - `18 <= Age < 65`

```
CREATE TABLE staples (  
    ID int NOT NULL,  
    Name varchar(255),  
    Type varchar(255),  
    Color varchar(255),  
    Price float,  
    PRIMARY KEY (ID),  
    CHECK (Price>=20)  
);
```

- Let's say we want to create a table with a column Price and this columns should only accept values that are bigger than 20
- In this case we could use the method shown on the left side

- When introducing CHECK constraints to SQL databases they will always be assigned a name, even if we don't specify a name for them
- Since it can be troublesome to get rid of such a CHECK constraint later on if we don't know the name of it, it's better to always name our CHECK constraints
- We can do this while creating our tables as well as adding them to a table later on (Next slides)

- To add a named CHECK constraint to a table while using the CREATE TABLE statement we need to adjust the syntax a bit
- Instead of writing only CHECK we need the statement CONSTRAINT followed by the name of our constraint and then the remaining CHECK statement:

```
CREATE TABLE staples (  
    ID int NOT NULL PRIMARY KEY,  
    Name varchar(255),  
    Type varchar(255),  
    Color varchar(255),  
    Price float,  
    CONSTRAINT Price_Check CHECK (Price>=20));
```

- As shown earlier we can also specify a range of values we want to allow in a specific column of our table:

```
CREATE TABLE staples (  
    ID int NOT NULL PRIMARY KEY,  
    Name varchar(255),  
    Type varchar(255),  
    Color varchar(255),  
    Price float,  
    CONSTRAINT CHK_Color CHECK (Color in ('W','U','B','R','G','C','L'))  
);
```

- The DEFAULT constraint is used to set a default value for a column
- The default value will be added to all new records, if no other value is specified when inserting data later on
- We add this constraint to a table when using the CREATE TABLE statement by simply writing DEFAULT and the value we want to use after the datatype of the corresponding column (next slide)

- Let's say we want to define a default value for a column called manufacturer in a table called sleeves
- We can do this during the creation of our table as shown below
- Don't forget that for string values we always need the quotes

```
CREATE TABLE sleeves (  
    ID int NOT NULL,  
    Type varchar(255),  
    Subtype varchar(255),  
    manufacturer varchar(255) DEFAULT 'Ultra Pro'  
);
```

- You saw how to add constraints to your table during the construction of these tables using CREATE TABLE
- You can also add Constraints retroactively to your tables using ALTER TABLE
- Be careful that the column you want to add the constraint to, has only values that fulfill the corresponding constraint
- Different Constraint make use of different forms of ALTER TABLE statements (next slides)

- You can add the NOT NULL constraint by using ALTER TABLE in combination with MODIFY:

```
ALTER TABLE decks MODIFY ID int NOT NULL;
```

- In a similar manner you can add the UNIQUE constraint:

```
ALTER TABLE decks MODIFY ID int UNIQUE;
```

- UNIQUE can also be added to your table using ADD:

```
ALTER TABLE decks ADD UNIQUE (FullName);
```

- You always need this version when using multiple columns for UNIQUE:

```
ALTER TABLE decks ADD UNIQUE (ID, Shorts);
```

- The PRIMARY KEY constraint can either be added using MODIFY

```
ALTER TABLE decks MODIFY ID int PRIMARY KEY;
```

- Alternatively you can use ADD, which you always need when defining a composite PRIMARY KEY:

```
ALTER TABLE boxes ADD PRIMARY KEY (ID);
```

```
ALTER TABLE boxes ADD PRIMARY KEY (ID, Type);
```

- The AUTO INCREMENT feature can be added using MODIFY:

```
ALTER TABLE sleeves MODIFY ID int NOT NULL Auto_Increment;
```

- The CHECK constraint can be added using the ADD statement:

```
ALTER TABLE staples ADD CHECK (Price>=20);
```

- If you want to name the constraint you need to use ADD CONSTRAINT:

```
ALTER TABLE decks ADD CONSTRAINT CHK_Price CHECK (Price>0);
```

```
ALTER TABLE staples ADD CONSTRAINT CHK_Color  
CHECK (Color in ('W','U','B','R','G','C','L'));
```

- To add a FOREIGN KEY you need the ADD FOREIGN KEY statement:

```
ALTER TABLE decks ADD FOREIGN KEY (Box_ID) REFERENCES boxes (ID);
```

- When working with ALTER TABLE we have two ways of adding default values to a column
- We can either use SET DEFAULT:

```
ALTER TABLE sleeves ALTER Manufacturer SET DEFAULT 'Dragon Shield';
```

- or we can use MODIFY and a new column definition:

```
ALTER TABLE sleeves MODIFY Type varchar(255) DEFAULT "Matte";
```

- Create 2 new databases called laboratory and greenhouses respectively. The database laboratory needs to contain the following tables:
 - Employees, Tools, Experiments, Results, Experiment_Results, Experiment_Employee, Experiment_Tools
- The database greenhouse needs to contain the following tables:
 - Greenhouses, Gardeners, Plants, Gardeners_Greenhouse, Plants_Greenhouse
- Make sure to include all necessary Primary and Foreign Keys
- Include each other Constraint at least once in each of the two databases
- You can find the column names in ABI_Exercises.pdf
- Look at the file ABI_SQL-Data before determining the needed keys

Inserting Data

- The INSERT INTO statement is used to insert new records in a table
- It is possible to write the INSERT INTO statement in two ways:

1. Specify both the column names and the values to be inserted:

```
INSERT INTO table_name (column1, column2, column3, ...)
```

```
VALUES (value1, value2, value3, ...);
```

2. If you are adding values for all the columns of the table, you do not need to specify the column names in the SQL query. However, make sure the order of the values is the same as the order of the columns in the table:

```
INSERT INTO table_name
```

```
VALUES (value1, value2, value3, ...);
```

- When using INSERT INTO you will encounter errors as soon as your data does not fit the rules for your table:
 - ▶ If you want to insert a text in a column defined with a datatype of int, float or decimal for example so this will not work
 - ▶ If you have a column with the NOT NULL constraint and you don't specify a value for it with INSERT INTO you will get also an error telling you that a certain constraint was not fulfilled
 - ▶ Here naming certain constraints comes in handy since it's easier to understand the error messages in these cases
- IN SUMMARY: Make sure that your data fits your constraints and vice versa

- When filling tables with data we need to make sure that our foreign keys can work correctly
- Therefore we must start by filling the parent tables of our relations first
- Afterwards we are able to fill the corresponding child tables, the ones with the foreign keys
- BE AWARE: a column that carries a foreign key can only be filled either with a NULL value or a value that is present in the corresponding primary key column

- To fill the tables in my database I need to add data to my parent table boxes first, so that later on I can fill the child table decks
- Here I simply add one dataset / row to my table and fill all four columns:

```
INSERT INTO boxes VALUES (5, 'Satin Cube', 'Pacific Blue', 'Ultra Pro');
```

- Now I'm able to enter data into my child table decks by specifying a value for each column in the table
- When doing so make sure the values in brackets follow the same order as the columns in the table:

```
INSERT INTO decks VALUES (1, 'Aeagar', 'UR', 'Aegar, the freezing  
Flame', 9, 209.30, 5);
```

- If you don't have data for all columns you can specify only certain columns and add only values for these columns:

```
INSERT INTO decks (ID, Short, Colors, FullName, Box_ID)
VALUES (2, 'Yoshimaru', 'GWB', 'Yoshimaru, Ever Faithful', 5);
```

- When using this method you can change the order of the columns as you want, as long as the order of the values matches the order of the columns you specified in your statement:

```
INSERT INTO decks (Short, Box_ID, ID, FullName, Colors)
VALUES ('Arix', 5, 3, 'Arixmethes, the slumbering Isle', 'UG');
```

- Be aware that you **ALWAYS** need to specify a foreign key column if present

- When working with tables that allow NULL values we can use the statement NULL while using INSERT INTO to leave a field empty:

```
INSERT INTO boxes VALUES (1,NULL,'Light Blue','Gamegenic');
```

- On the other hand if you specified a DEFAULT value for a column in your table you can use the statement DEFAULT instead of a value while using INSERT INTO:

```
INSERT INTO sleeves VALUES (2,DEFAULT,'Pacific Blue',DEFAULT);
```

- Use the INSERT INTO method to fill your tables
- Use the data in the file ABI_SQL-Data

UPDATE

- The UPDATE statement is used to modify the existing rows in a table, that means we can use it to change the data in our table
- We can use UPDATE only in combination with a condition and these conditions need to refer to Primary or Foreign key columns
- Syntax:

UPDATE table_name

SET column1 = value1, column2 = value2, ...

WHERE condition;

- We can change the values of the data in the columns Short, FullName and Turn for our table decks as shown below
- Here I use the primary Key of my table in the condition and therefore I'm able to run this statement:

```
UPDATE decks  
  
SET Short="Hello", FullName="Hello World", Turn=8  
  
WHERE ID=1;
```

- On the other hand I could use a foreign key column in my condition as well:

```
UPDATE decks SET Turn=7 WHERE Box_ID=5;
```

- On the other hand I'm not able to use non-key columns, that means either primary or foreign key columns, in my condition to run an UPDATE statement
- So the example below does NOT work:

```
UPDATE decks SET Turn=2 WHERE FullName LIKE '%S';
```

- Use the UPDATE statement to make changes in the database laboratory:
 - ▶ Raise the Salary of Gundolf Stoll to 3250
 - ▶ Change the Manufacturer of Big Tube to HelloResearch and decrease the price to 0,90€
 - ▶ Reduce the time of DNA-Seq to 210
 - ▶ Increase the repeats for the experiment on Monday to 12 and the time to 180
- Use the UPDATE statement to make changes in the database greenhouse:
 - ▶ Decrease the salary of Anna to 4800
 - ▶ Set the size of the greenhouse in Bush Road 5 to 450, #Patches to 180, the temperature to 33°C and the humidity to 72.5%
 - ▶ Change the mutation of TomatoCG to C500G

DELETE

- The DELETE statement is used to delete existing rows in a table
- Again we can use DELETE only in combination with conditions and these conditions need to follow the same rules as the ones for UPDATE
- Syntax:

```
DELETE FROM table_name WHERE condition;
```

- Example:

```
DELETE FROM boxes WHERE ID=3;
```

- In the Laboratory Gundolf Stoll went into retirement, so all data related to him has to be removed from the database
- The Lab on a Chip is too expensive so remove all entries related to it as well
- The smallest greenhouse is not rentable enough, so it was sold
 - ▶ Remove all entries that deal with it from the database

- The Database Management part contains some additional exercises
 - Exercise 7 to Exercise 10
- Try your hands on creating these databases with the help of the MySQL workbench
- Don't use the Translation help of ERD-Plus

Definitions

- We use **Structured Query Language (SQL)** to communicate with databases and manipulate them and the data they save
- The definition of a column is called **HEADER** in SQL
- A **Field** in a table is the smallest unit of a table and contains the value associated to one row and one column at the same time
- We use **SQL Statements** to hand over commands to our database
- We use the SELECT statement to create **Selections** of our data
- A SELECT statement inside the condition of another SELECT statement is called **Sub Query**
- In MySQL a database is called a **SCHEMA**

1. <https://www.w3schools.com/sql/> (Last visited: 12.01.2023)
2. <https://en.wikipedia.org/wiki/SQL> (Last edited: 21.04.2023, Last visited: 27.04.2023)
3. https://www.w3schools.com/mysql/mysql_datatypes.asp (Last visited: 28.04.2023)
4. <https://planetscale.com/learn/courses/mysql-for-developers/queries/unions> (Last edited: 08.11.2023, Last visited: 23.01.2024)
5. <https://phoenixnap.com/kb/mysql-stored-procedure> (Last edited: 13.04.2021, Last visited: 23.01.2024)